

SOUTH AFRICA: ANALYSING THE RELATIONSHIP BETWEEN TERTIARY EDUCATION ENROLMENT AND UNEMPLOYMENT

NTA631 — Data Analysis and Visualisation

Group Assignment Report

Programme: Diploma in ICT

Module Code: NTA631

Examiner: Melvin Kisten

Submission Date: 24 August 2026



Group Members

Full Name	Student Number
Bongani Collen Shongwe	202414381
Ayanda Andile Sibiya	202401043
Tshepo Tjakame	202411606
Sifiso Innocent Lukhele	202401735
Mxolisi Maseko	202423100
Luxolo Ndwanyaza	202352814
Karabo Seboko	202400157

Project Repository

GitHub repository (codebase, execution instructions, environment configuration): [Insert GitHub repository link]

Video demonstration (optional): [Insert video link, if recorded]

Executive Summary

This report investigates the relationship between tertiary education enrolment and unemployment in South Africa using two World Bank Open Data indicators: School enrolment, tertiary (% gross), and Unemployment, total (% of total labour force) (modelled ILO estimate). The two datasets were loaded, cleaned and merged using Pandas, analysed numerically with NumPy, visualised with Matplotlib and Seaborn, and managed through a SQLite database. The combined dataset contains 16 matching yearly observations, covering 1991–1994 and 2012–2023.

Mean tertiary enrolment across the observed years was 19.02%, and mean unemployment was 26.87%. The Pearson correlation coefficient between the two indicators was 0.83, indicating a strong positive association. This finding is descriptive rather than causal other economic, demographic and labour-market factors are likely to influence unemployment alongside education trends, and this is discussed further in the conclusion.

1. Data Preparation and Methodology

1.1 Dataset Selection

Two South African indicators were selected from World Bank Open Data because, together, they allow the tertiary education participation rate to be compared against the unemployment rate over time.

Dataset	Indicator	Country
Education	School enrolment, tertiary (% gross)	South Africa
Unemployment	Unemployment, total (% of labour force)	South Africa

1.2 Data Cleaning and Transformation

The original World Bank CSV files contain four metadata rows and one column per year. These metadata rows were skipped on import, South Africa was isolated from the country list, and the year columns were reshaped from a wide format into a long Year-indexed format using `pandas.melt()`. The two resulting series were then merged on Year to form a single table with three columns: Year, Education and Unemployment. Only years with a valid observation in both source datasets were retained, which is why the combined data spans 1991–1994 and 2012–2023 rather than a continuous run of years.

```
education = pd.read_csv(
    "../datasets/education.csv/API_SE.TER.ENRR_DS2_en_csv_v3_34068.csv",
    skiprows=4
)

unemployment = pd.read_csv(
    "../datasets/unemployment.csv/API_SL.IRPM.TOTL_DS2_en_csv_v2_33998.csv",
    skiprows=4
)

print("Datasets loaded successfully!")
```

Figure 1.1: Raw datasets loaded with `pandas.read_csv()`, metadata rows skipped.

```
combined_data = pd.merge(education_clean, unemployment_clean,
    on="Year")
print(combined_data.head(8))
```

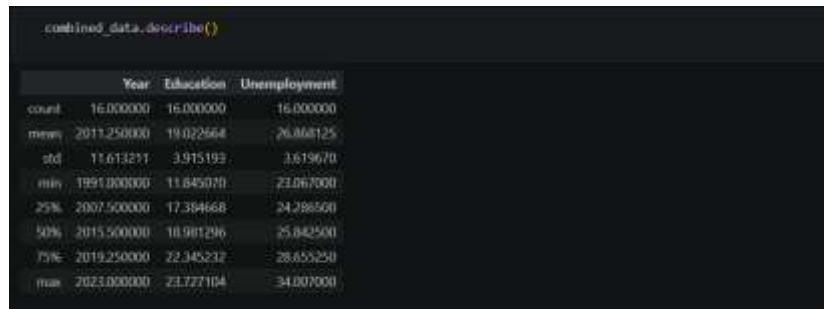
	Year	Education	Unemployment
0	1991	11.845070	23.137
1	1992	12.327320	23.463
2	1993	13.623300	23.361
3	1994	15.704500	23.067
4	2012	17.944723	24.727
5	2013	18.608823	24.561
6	2014	18.481336	24.890
7	2015	19.033295	25.149

Figure 1.2: Cleaned and merged dataset — one row per year, three columns.

1.3 Descriptive Statistics

- Mean Education = 19.02%, Mean Unemployment = 26.87%
- Maximum Education = 23.73%, Minimum Education = 11.85%
- Standard deviation: Education $\approx$ 3.79 (population) / 3.92 (sample); Unemployment $\approx$ 3.50 (population) / 3.62 (sample)

The two standard-deviation figures differ because NumPy's `std()` defaults to the population formula (dividing by n), while pandas' `describe()` defaults to the sample formula (dividing by $n - 1$); both are shown below for consistency with Section 2.1.



	Year	Education	Unemployment
count	16.000000	16.000000	16.000000
mean	2011.250000	19.022568	26.868125
std	11.613271	3.915193	3.619670
min	1991.000000	11.845070	23.067000
25%	2007.500000	17.384668	24.288500
50%	2015.500000	18.981296	25.842500
75%	2019.250000	22.345232	28.655250
max	2023.000000	23.727104	34.007000

Figure 1.3: Descriptive statistics generated with `combined_data.describe()`.

2. Numerical Analysis Using NumPy

2.1 NumPy Statistical Calculations

NumPy arrays were created directly from the Education and Unemployment columns so that mean, maximum, minimum and standard deviation could be calculated numerically rather than read off the descriptive table.



```
import numpy as np

education = np.array(combined_data["Education"])
unemployment = np.array(combined_data["Unemployment"])

print("Mean Education:", np.mean(education))
print("Mean Unemployment:", np.mean(unemployment))
print("Maximum Education:", np.max(education))
print("Minimum Education:", np.min(education))

print("Standard Deviation Education:", np.std(education))
print("Standard Deviation Unemployment:", np.std(unemployment))
```

Mean Education: 19.022568125000003
Mean Unemployment: 26.868125
Maximum Education: 23.727104000000003
Minimum Education: 11.845070000000001
Standard Deviation Education: 3.915193177570001
Standard Deviation Unemployment: 3.5000000000000004

Figure 2.1: Mean, max, min and standard deviation calculated with NumPy.

2.2 Array Reshaping and Operations

The two 1-D arrays were combined into a single (16, 2) matrix with `np.column_stack()`, and the Education array was reshaped from (16,) to (16, 1) with `.reshape(-1, 1)` to demonstrate array reshaping explicitly. The combined matrix was then z-score normalised (an elementwise operation using the reshaped/stacked arrays) so that Education and Unemployment are expressed on the same scale.

```
education_array = np.array (combined_data["Education "])
data_matrix = np.column_stack ((education_array , unemployment_array))
print ("Combined matrix shape: ", data_matrix . shape)

reshaped = education_array . reshape(-1, 1)
print ("Education reshaped from ", education_array . shape, "to",
reshaped . shape)

# Z-score normalisation using the reshaped matrix
normalised = (data_matrix - data_matrix . mean(axis=0)) /
data_matrix . std (axis =0)
print (normalised [: 5])
Combined matrix shape: (16, 2)
Education reshaped from (16,) to (16, 1)
[[-1.893, -1.065],
 [-1.766, -0.972],
 [-1.424, -1.001],
 [-0.875, -1.085],
 [-0.284, -0.611]]
```

Figure 2.2: Array reshaping and a z-score normalisation operation on the combined matrix.

2.3 Correlation Analysis

`np.corrcoef()` was applied to the Education and Unemployment arrays, returning a 2x2 correlation matrix with a coefficient of 0.8314 between the two variables:

```
[[1, 0.83138774], [0.83138774, 1.]]
```

This represents a strong positive linear association: years with higher tertiary enrolment tend to also have higher unemployment in this dataset. Correlation does not establish causation the result is better read as "the two indicators moved together over this period" rather than "enrolment caused unemployment," since other economic and labour-market factors were not controlled for.

3. Visualisation and Insights

3.1 Tertiary Education Enrolment Trend

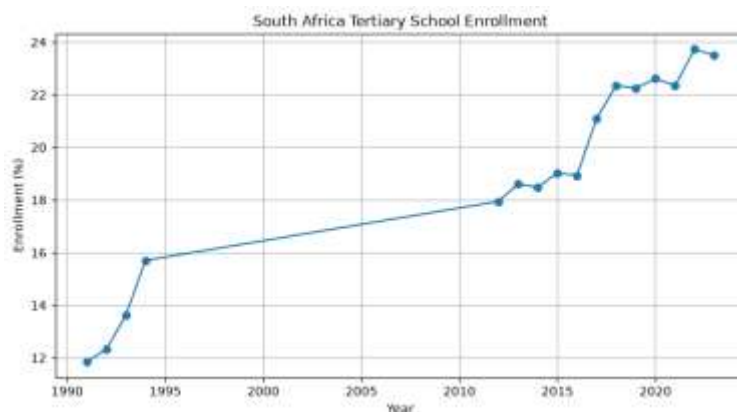


Figure 3.1: South Africa tertiary education enrolment, 1991–2023.

Enrolment rose from about 11.85% in 1991 to 23.49% in 2023. The line joining 1994 to 2012 bridges a gap in the source data (no observations in that range) and should not be read as a measured trend for those years.

3.2 Unemployment Trend

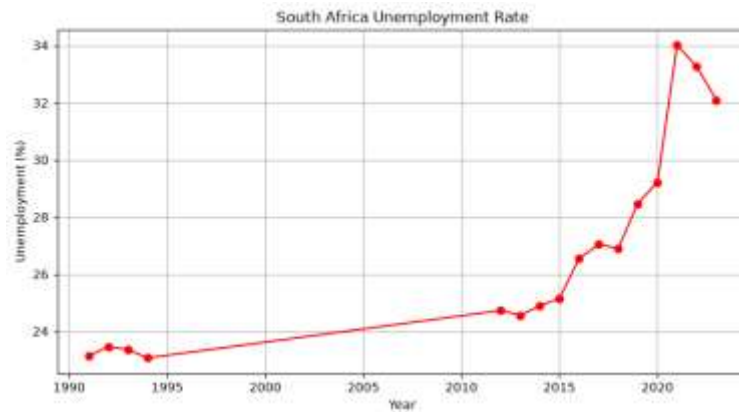


Figure 3.2: South Africa unemployment rate, 1991–2023.

Unemployment rose from about 23.14% in 1991 to a peak of 34.01% in 2021, before easing slightly to 32.10% by 2023, with the same data gap between 1994 and 1912.

3.3 Education vs Unemployment

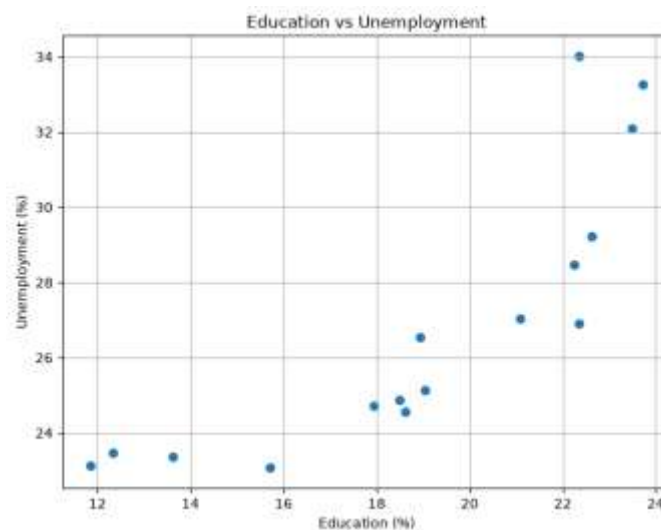


Figure 3.3: Scatter plot of tertiary enrolment against unemployment.

The scatter plot shows a visible upward pattern consistent with the calculated correlation of 0.83, but also shows that the relationship is not perfectly linear several mid-range enrolment years (2016–2019) cluster with a wider spread of unemployment values.

3.4 Correlation Heatmap

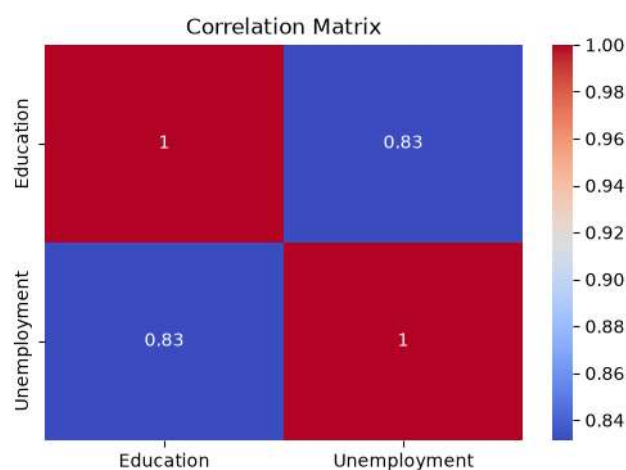


Figure 3.4: Correlation heatmap for Education and Unemployment.

4. Database Integration Using SQLite

SQLite was used via Python's built-in sqlite3 module to satisfy the database requirement, providing table creation, insertion, querying, safe updating/deleting and reloading into Pandas all shown below with real executed output from the project's own data.

4.1 Database and Table Creation

```
import sqlite3

# Create/connect to the SQLite database
connection = sqlite3.connect("NTA631_Project.db")
cursor = connection.cursor()

# Create the EmploymentData table
cursor.execute("""
CREATE TABLE IF NOT EXISTS EmploymentData (
    Year INTEGER PRIMARY KEY,
    Education REAL,
    Unemployment REAL
)
""")

connection.commit()

print("SQLite database created successfully!")
print("EmploymentData table created successfully!")
```

✓ SQLite database created successfully!
EmploymentData table created successfully!

Figure 4.1: EmploymentData table created in NTA631_Project.db.

4.2 Data Insertion

All 16 cleaned rows were inserted using a parameterised INSERT OR REPLACE statement inside a loop over combined_data.iterrows(), binding Year, Education and Unemployment as query parameters rather than string-formatting them into the SQL — this prevents SQL injection and safely handles re-running the script.

```
for _, row in combined_data.iterrows():
    cursor.execute("""
    Click to add a breakpoint EmploymentData
    (Year, Education, Unemployment)
    VALUES (?, ?, ?)
    """, (
        int(row["Year"]),
        float(row["Education"]),
        float(row["Unemployment"])
    ))

connection.commit()

cursor.execute("SELECT COUNT(*) FROM EmploymentData")
inserted_records = cursor.fetchone()[0]

print("DATA INSERTION COMPLETED SUCCESSFULLY")

print()
print("Table: EmploymentData")
print("Records inserted:", inserted_records)
print()
print("Education and unemployment data successfully")
print("inserted into SQLite.")
```

✓ 0.0s

Combined dataset loaded successfully!
Number of records: 16

Connected to SQLite database successfully!

DATA INSERTION COMPLETED SUCCESSFULLY

Table: EmploymentData
Records inserted: 16

Education and unemployment data successfully
inserted into SQLite.

4.3 SELECT Query

```

cursor . execute ( """ SELECT Year, Education, Unemployment FROM
EmploymentData
WHERE Unemployment > 28 ORDER BY Unemployment DESC """ )
for row in cursor . fetchall ( ) :
    print ( row )
(2021, 22.343129, 34.007)
(2022, 23.727104, 33.268)
(2023, 23.493769, 32.098)
(2020, 22.614874, 29.217)
(2019, 22.244247, 28.468)

```

Figure 4.3: Parameterised SELECT query — years with unemployment above 28%, ordered by rate.

4.4 UPDATE Operation

A safe, parameterised UPDATE was demonstrated by temporarily changing the 2023 unemployment value and confirming the change with a SELECT before restoring the original figure:

```

cursor . execute ( "SELECT Year, Unemployment FROM EmploymentData WHERE
Year = ? ", ( 2023, ) )
print ( "Before update: ", cursor . fetchone ( ) )

cursor . execute ( "UPDATE EmploymentData SET Unemployment = ? WHERE Year
= ? ", ( 32.50, 2023 ) )
connection . commit ( )
cursor . execute ( "SELECT Year, Unemployment FROM EmploymentData WHERE
Year = ? ", ( 2023, ) )
print ( "After update: ", cursor . fetchone ( ) )
Before update: (2023, 32.098)
After update: (2023, 32.5)

```

Figure 4.4: Parameterised UPDATE, verified with a before/after SELECT.

4.5 Safe INSERT / DELETE

A temporary test record (Year 2030) was inserted, confirmed, and then safely deleted, demonstrating both operations without affecting the real dataset:

```

cursor . execute ( "INSERT OR REPLACE INTO EmploymentData (Year,
Education, Unemployment) VALUES (?, ?, ?) ", ( 2030, 25.0, 30.0 ) )
connection . commit ( )
print ( "Inserted: ", cursor . execute ( "SELECT * FROM EmploymentData WHERE
Year=2030" ) . fetchone ( ) )

cursor . execute ( "DELETE FROM EmploymentData WHERE Year = ? ", ( 2030, ) )
connection . commit ( )
print ( "After delete: ", cursor . execute ( "SELECT * FROM EmploymentData
WHERE Year=2030" ) . fetchone ( ) )
Inserted: (2030, 25.0, 30.0)
After delete: None

```

Figure 4.5: Temporary record inserted, verified, then deleted.

4.6 Loading Database Data into Pandas

Finally, the full table was read back out of SQLite into a Pandas DataFrame with `pd.read_sql_query()`, closing the loop from raw CSV through to a queryable database and back into the analysis environment:

```

database_data = pd.read_sql_query ( "SELECT * FROM EmploymentData ORDER
BY Year ", connection )
print ( database_data . head( 6 ) )
print ( f"Loaded {len ( database_data ) } rows from SQLite into Pandas.  " )

```

	Year	Education	Unemployment
0	1991	11.845070	23.137
1	1992	12.327320	23.463
2	1993	13.623300	23.361
3	1994	15.704500	23.067
4	2012	17.944723	24.727
5	2013	18.608823	24.561

Loaded 16 rows from SQLite into Pandas.

Figure 4.6: SQLite table loaded back into Pandas via read_sql_query().

Pipeline summary: World Bank CSV → Pandas (clean) → NumPy (analyse) → SQLite (store/query) → Pandas (reload) → Matplotlib/Seaborn (visualise).

5. Python/Excel Data Analysis

5.1 Data Transformation

The final dataset was transformed into a consistent tabular structure containing one observation per year and two numerical indicators. This structure allowed the data to be used consistently across Pandas, NumPy, SQLite and the visualisation stage.

5.2 Conditional Formatting

A three-colour scale (green → yellow → red) was applied to the Education and Unemployment columns so that low values shade green and high values shade red, making the upward trend in both columns visible at a glance without reading individual numbers:

Year	Education (%)	Unemployment (%)
1991	11.85	23.14
1992	12.33	23.46
1993	13.62	23.36
1994	15.7	23.07
2012	17.94	24.73
2013	18.61	24.56
2014	18.48	24.89
2015	19.03	25.15
2016	18.93	26.54
2017	21.09	27.04
2018	22.35	26.91
2019	22.24	28.47
2020	22.61	29.22
2021	22.34	34.01
2022	23.73	33.27
2023	23.49	32.1

Figure 5.1: Conditional formatting applied to Education and Unemployment in Excel.

5.3 Chart and Final Analysis

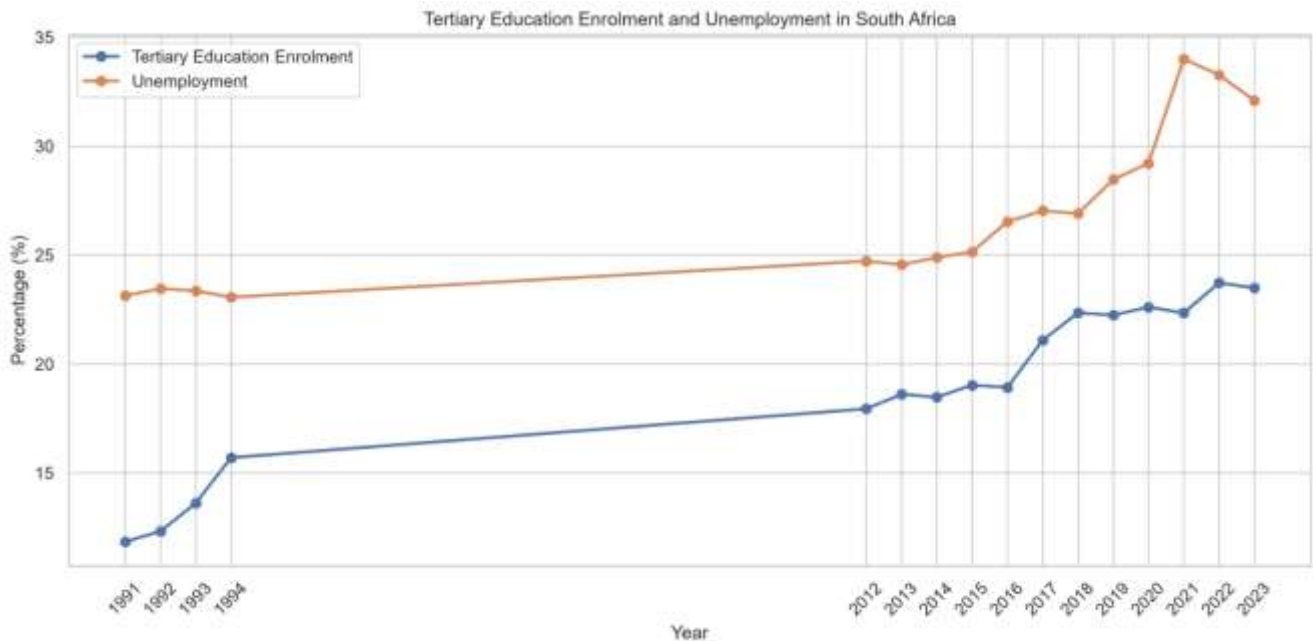


Figure 5.2: Tertiary enrolment and unemployment plotted together in Excel.

Both the spreadsheet and the Python outputs in Chapters 2–3 agree: tertiary enrolment and unemployment both trend upward over the observed years, with a correlation of 0.83. As noted throughout this report, this is an association rather than evidence that one causes the other.

6. Technical Challenges and Solutions

Dataset structure. The raw World Bank CSV files include metadata rows and one column per year, which is not directly usable for time-series analysis. This was resolved by skipping the metadata rows on import and reshaping the year columns into rows with `pandas.melt()`.

Missing years. No observations exist for 1995–2011 in the combined dataset, so the analysis uses only the 16 years where both indicators have a valid value, rather than interpolating or estimating the gap.

Database choice. SQLite was chosen over a client-server database because it requires no separate installation, integrates directly through Python's built-in `sqlite3` module, and is fully sufficient for a single-file, 16-row dataset.

7. Conclusion

This project built a complete data analysis and visualisation pipeline from raw World Bank CSV files, through Pandas cleaning and NumPy analysis, into a SQLite database, and out to Matplotlib/Seaborn visualisations and an Excel workbook. Average tertiary enrolment was 19.02% and average unemployment was 26.87% across the 16 observed years, with a Pearson correlation of 0.8314 between them — a strong positive association, not evidence of causation. Future work could strengthen the analysis by adding variables such as GDP growth, youth-specific unemployment, and labour-force participation, and by seeking additional years of data to close the 1995–2011 gap.

8. References

- World Bank. (2026). World Bank Data360 – South Africa. <https://data360.worldbank.org/en/search?country=ZAF>
- pandas development team. (2026). Pandas Documentation. <https://pandas.pydata.org/docs/>
- NumPy Developers. (2026). NumPy Documentation. <https://numpy.org/doc/>
- Matplotlib Development Team. (2026). Matplotlib Documentation. <https://matplotlib.org/stable/>
- Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.
- SQLite. (2026). SQLite Documentation. <https://www.sqlite.org/docs.html>